Yolov11 Garbage Sorting

1. Function Description

After the program starts, the camera captures images. Place the garbage tag block in the image. The robotic arm recognizes the category of the garbage tag block and lowers the gripper to grip the garbage tag block. Based on the recognized category of the garbage tag, it places it at the set position. After placement is complete, it returns to the recognition pose.

2. Startup and Operation

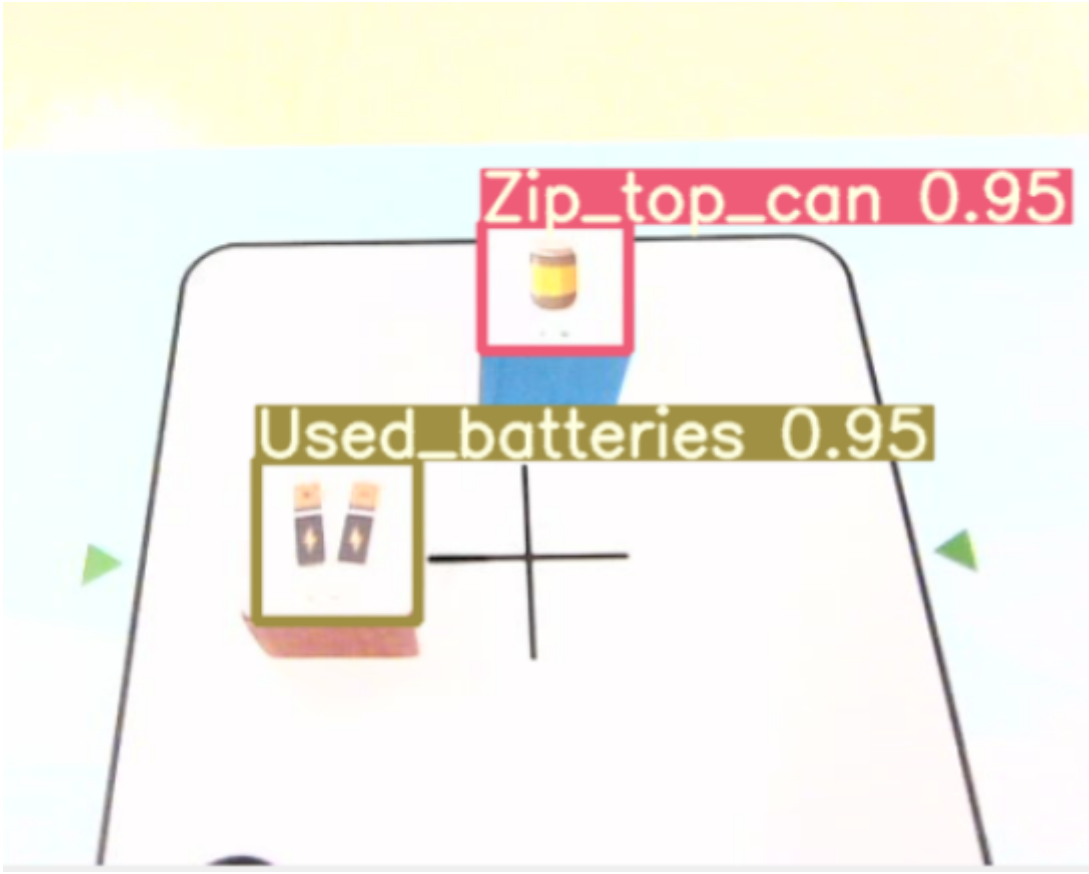
2.1 Startup Commands

Enter the following commands in the terminal to start:

```
# Start camera and inverse kinematics program
ros2 launch dofbot_info camera_arm_kin.launch.py
# Start robotic arm garbage sorting program:
ros2 run dofbot_sorting_3d yolov11_garbage
# Start robotic arm gripping program
ros2 run dofbot_sorting_3d grasp
```

2.2 Operation Process

After the program starts, place the garbage tag block in the center of the image. The block must be aligned properly. Press the spacebar to start recognition. The robotic arm will, based on the recognized garbage category, lower the gripper to grip the block and place it at the corresponding set position.



4. Core Code Analysis

4.1.1 yolov11_garbage.py

Code path:

```
/home/yahboom/dofbot_ws/src/dofbot_sorting_3d/dofbot_sorting_3d/yolov11_garbage.
py
```

Import necessary libraries,

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
import rclpy
from rclpy.node import Node
import cv2
import numpy as np
from sensor_msgs.msg import Image
from message_filters import ApproximateTimeSynchronizer, Subscriber
from std_msgs.msg import Float32, Bool
from cv_bridge import CvBridge
import cv2 as cv
from dt_apriltags import Detector
import transforms3d as tfs
import tf_transformations as tf         # ROS2 uses tf_transformations
import threading

from dofbot_interface.srv import Kinemarics
from dofbot_interface.msg import *
import pyzbar.pyzbar as pyzbar
from std_msgs.msg import Float32,Bool,Int16
import time
import queue
import math
from Arm_Lib import Arm_Device
from ultralytics import YOLO
import warnings
```

Initialize program parameters, create publishers, subscribers,

```
# Initialize the node with name 'apriltag_detect'
# Initialize node with name 'apriltag_detect'
def __init__(self):
    super().__init__('apriltag_detect')

    # Create an instance of the arm control device
    # Create arm control device instance
    self.Arm = Arm_Device()

    # Initial joint angles configuration
    # Initial joint angles configuration (unit: degrees)
    self.init_joints = [90.0, 120.0, 0.0, 0.0, 90.0, 0.0]

    # Variable to store target angle for joint (Int16 type)
    # Variable to store target angle for joint (Int16 type)
    self.joint5 = Int16()
```

```
    # Detection flag to control whether detection is active
    # Detection flag to control whether detection is active
    self.detect_flag = False

    # Switch for height computation
    # Switch for height computation
    self.compute_height = True

    # Index value for recording or locating target
    # Index value for recording or locating target
    self.index = None

    # Create a client for kinematics service (type: Kinemarics, name:
'dofbot_kinemarics')
    # Create kinematics service client
    self.client = self.create_client(Kinemarics, 'dofbot_kinemarics')

    # Publisher: publish target angle of joint 5 to "set_joint5" topic
    # Publisher: publish target angle of joint 5 to "set_joint5" topic
    self.TargetJoint5_pub = self.create_publisher(Int16, "set_joint5", 10)

    # Publisher: publish target position info to "PosInfo" topic
    # Publisher: publish target position info to "PosInfo" topic
    self.pos_info_pub = self.create_publisher(AprilTagInfo, "PosInfo",
qos_profile=10)

    # Subscriber: listen to "grasp_done" topic, callback function is
GraspStatusCallback
    # Subscriber: listen to "grasp_done" topic, callback function is
GraspStatusCallback
    self.subscription = self.create_subscription(Bool, 'grasp_done',
self.GraspStatusCallback, qos_profile=1)

    # Bridge to convert ROS RGB image messages to OpenCV format
    # Bridge to convert ROS RGB image messages to OpenCV format
    self.rgb_bridge = CvBridge()

    # Bridge to convert ROS depth image messages to OpenCV format
    # Bridge to convert ROS depth image messages to OpenCV format
    self.depth_bridge = CvBridge()

    # Flag to enable/disable position publishing
    # Flag to enable/disable position publishing
    self.pubPos_flag = False

    # Record timestamp of last processing
    # Record timestamp of last processing
    self.pr_time = time.time()

    # Create AprilTag detector with the following parameters:
    # Create AprilTag detector with the following parameters:
    self.at_detector = Detector(
        searchpath=['apriltags'],      # Search path
                                       # Search path for tag files
        families='tag36h11',           # Tag family used
                                       # Tag family used
        nthreads=8,                    # Number of threads
```

```
                                    # Number of threads
        quad_decimate=2.0,          # Quadrilateral decimation factor
                                    # Quadrilateral decimation factor
        quad_sigma=0.0,             # Gaussian blur sigma
                                    # Gaussian blur sigma
        refine_edges=1,             # Edge refinement switch
                                    # Edge refinement switch
        decode_sharpening=0.25,     # Decoding sharpening factor
                                    # Decoding sharpening factor
        debug=0                     # Debug mode switch
                                    # Debug mode switch
    )

    # Target AprilTag ID
    # Target AprilTag ID
    self.target_id = 31

    # List to store detected center X coordinates
    # List to store detected center X coordinates
    self.Center_x_list = []

    # List to store detected center Y coordinates
    # List to store detected center Y coordinates
    self.Center_y_list = []

    # Initialize arm joints to initial positions, duration 2000ms
    # Initialize arm joints to initial positions, duration 2000ms
    self.Arm.Arm_serial_servo_write6_array(self.init_joints, 2000)

    # Default distance parameter
    # Default distance parameter (unit: meters), used for coordinate conversion
    self.dist = 0.13

    # Subscribe to RGB camera raw image topic "/image_raw", callback is
ImageCallback
    # Subscribe to RGB camera raw image topic "/image_raw", callback is
ImageCallback
    self.subscription = self.create_subscription(Image, '/image_raw',
self.ImageCallback, 1)

    # Start flag to control whether to process images
    # Start flag to control whether to process images
    self.start_flag = True

    # List of recyclable waste categories
    # List of recyclable waste categories
    self.recyclable_waste = ['Newspaper', 'Zip_top_can', 'Book',
'Old_school_bag']

    # List of toxic waste categories
    # List of toxic waste categories
    self.toxic_waste = ['Syringe', 'Expired_cosmetics', 'Used_batteries',
'Expired_tablets']

    # List of wet waste categories
    # List of wet waste categories
    self.wet_waste = ['Fish_bone', 'Egg_shell', 'Apple_core', 'Watermelon_rind']
```

```
    # List of dry waste categories
    # List of dry waste categories
    self.dry_waste = ['Toilet_paper', 'Peach_pit', 'Cigarette_butts',
'Disposable_chopsticks']
```

ImageCallback callback function,

```
# Convert ROS image message to OpenCV format (RGB8)
# Convert ROS image message to OpenCV format (RGB8)
rgb_image = self.rgb_bridge.imgmsg_to_cv2(color_frame, 'rgb8')

# Copy the RGB image for later processing
# Copy the RGB image for later processing
result_image = np.copy(rgb_image)
frame = np.copy(rgb_image)

# Run YOLO model inference
# Run YOLO model inference
results = model(frame, verbose=False)

# Wait for a key press (10 ms delay)
# Wait for a key press (10 ms delay)
key = cv2.waitKey(10)

# Enable position publishing when the space key is pressed
# Enable position publishing when the space key is pressed
if key == 32:
    self.pubPos_flag = True

# Get detection bounding boxes
# Get detection bounding boxes
boxes = results[0].boxes

# Process detections if they exist and the start flag is True
# Process detections if they exist and the start flag is True
if boxes != [None] and self.start_flag == True:
    for box in boxes:
        # Get bounding box top-left and bottom-right coordinates
        # Get bounding box top-left and bottom-right coordinates
        x_min, y_min, x_max, y_max = map(int, box.xyxy[0])

        # Get class ID and confidence score
        # Get class ID and confidence score
        class_id = int(box.cls)
        confidence = float(box.conf)

        # Create label string (class name + confidence)
        # Create label string (class name + confidence)
        label = f"{model.names[class_id]} {confidence:.2f}"

        # Calculate the center point of the bounding box
        # Calculate the center point of the bounding box
        center_x = (x_min + x_max) // 2
        center_y = (y_min + y_max) // 2

        # Convert pixel coordinates to robot coordinate system
        # Convert pixel coordinates to robot coordinate system
```

```
        (a, b) = (
            round(((center_x - 320) / 4000), 5),
            round(((480 - center_y) / 3000) * 0.8 + 0.13, 5)
        )

        # If position publishing is enabled
        # If position publishing is enabled
        if self.pubPos_flag == True:
            self.pubPos_flag = False  # Temporarily disable flag to prevent
continuous publishing
                                      # Temporarily disable flag to prevent
continuous publishing

            # Create an AprilTagInfo message object
            # Create an AprilTagInfo message object
            center = AprilTagInfo()
            center.x = b  # Set X coordinate
                          # Set X coordinate
            center.y = -a  # Set Y coordinate
                          # Set Y coordinate
            center.z = 0.03  # Set Z coordinate
                             # Set Z coordinate

            # Set corresponding ID based on waste category
            # Set corresponding ID based on waste category
            if str(model.names[class_id]) in self.recyclable_waste:
                center.id = 1  # Recyclable waste
                               # Recyclable waste
            elif str(model.names[class_id]) in self.wet_waste:
                center.id = 2  # Wet waste
                               # Wet waste
            elif str(model.names[class_id]) in self.toxic_waste:
                center.id = 3  # Toxic waste
                               # Toxic waste
            elif str(model.names[class_id]) in self.dry_waste:
                center.id = 4  # Dry waste
                               # Dry waste

            # Publish position information to ROS topic
            # Publish position information to ROS topic
            self.pos_info_pub.publish(center)

# Draw detection results on the image
# Draw detection results on the image
annotated_frame = results[0].plot()

# Convert RGB image to BGR format (required for OpenCV display)
# Convert RGB image to BGR format (required for OpenCV display)
annotated_frame = cv2.cvtColor(annotated_frame, cv2.COLOR_RGB2BGR)

# Resize and display the inference result window
# Resize and display the inference result window
cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

# Wait for a key press (1ms delay)
# Wait for a key press (1 ms delay)
key = cv2.waitKey(1)
```

4.1.2 grasp.py

The previous tutorial has explained this, so I won't repeat it here